Implementação de +10 Funcionalidades do Projeto

Nesta etapa, cada grupo deverá implementar e enviar ao repositório do projeto pelo menos 10 funcionalidades completas, seguindo obrigatoriamente a arquitetura apresentada nos materiais da disciplina.

Uma funcionalidade só será considerada implementada quando estiver disponível de ponta a ponta:

Interface/Tela → API Flask → Controller → Service → Model/Repository → Banco de Dados

Não contam como funcionalidades completas uma tela, uma rota, uma função isolada ou uma operação feita diretamente no banco de dados.

Requisitos obrigatórios

1. Frontend

- Cada funcionalidade deverá possuir uma interface que permita ao usuário executá-la.
- O frontend deverá consumir a API desenvolvida no backend.

2. Controllers

- As Controllers deverão ser implementadas como classes.
- Cada Controller deverá estar relacionada a um recurso ou Model do sistema.
- A Controller deverá receber a requisição HTTP, interpretar os dados recebidos, chamar o Service correspondente e retornar a resposta da API.
- Regras de negócio não deverão ser implementadas diretamente na Controller.

3. Services

- Cada caso de uso deverá possuir seu próprio Service. Exemplos: cadastrar usuário, atualizar produto, excluir pedido, buscar produtos por categoria ou gerar relatório.
- Cada Service deverá ser uma classe responsável por uma única ação do sistema.
- O Service deverá executar as validações e coordenar o caso de uso, chamando a Model ou o Repository quando necessário.

4. Models

- As Models deverão representar as entidades do domínio.
- Para o CRUD convencional, deverão herdar de db.Model.
- As operações básicas de persistência deverão permanecer na própria Model, quando aplicável:
  - salvar()
  - atualizar()
  - deletar()
  - listar_todos()
  - buscar_por_id()

5. Repository

- Não criem Repository apenas para repetir operações CRUD simples.
- Utilizem Repository para operações especiais de acesso aos dados, como filtros específicos, consultas complexas, relatórios, rankings ou chamadas a Stored Procedures.
- Se utilizarem uma Stored Procedure, a chamada CALL deverá ficar encapsulada no Repository. A Controller e o Service não deverão executar SQL nem chamar procedures diretamente.

6. Integrações com IA, APIs ou serviços externos

- Se uma funcionalidade utilizar Gemini, outra LLM ou uma API externa, a comunicação com o serviço deverá ficar encapsulada em um Service específico.
- Chaves de API e outras informações sensíveis não poderão ser publicadas no repositório. Utilizem variáveis de ambiente e arquivo .env.

Organização esperada

O projeto deverá apresentar uma separação clara de responsabilidades, seguindo a organização trabalhada em aula:

backend/
  controllers/
  services/
  models/
  repositories/

frontend/
  ...

Não concentrem toda a implementação em poucos arquivos. A organização das pastas e classes também faz parte da avaliação.

Diagrama de classes

O projeto deverá incluir um diagrama de classes do domínio que represente as classes implementadas nas Models e seus relacionamentos.

O diagrama deverá:

- apresentar as principais classes, atributos e relacionamentos do domínio;
- identificar, quando houver, herança, composição e agregação;
- ser coerente com as Models implementadas no backend e com a estrutura de dados do sistema;
- ser incluído no repositório, por exemplo, na pasta docs/ ou em outra localização indicada no README.

README.md

O README.md do repositório deverá conter uma seção chamada “Funcionalidades Implementadas”, enumerando claramente as 20 funcoinalidades do projeto ou mais funcionalidades entregues.

Exemplo:

1. Cadastrar cliente
2. Listar clientes
3. Atualizar cliente
4. Excluir cliente
5. Cadastrar produto
6. Listar produtos
7. Buscar produtos por categoria
8. Registrar pedido
9. Consultar histórico de pedidos
10. Gerar relatório de vendas

...

20. Recomendar produtos por gosto do usuário

A lista deverá corresponder às funcionalidades realmente implementadas e funcionando no código. O README também deverá indicar onde encontrar o diagrama de classes no repositório.

Entrega

Ao finalizar, o grupo deverá:

- realizar o commit e o push de todo o código para o repositório oficial do projeto;
- incluir no repositório o diagrama de classes;
- atualizar o README.md com a seção “Funcionalidades Implementadas”;
- enviar um vídeo de demonstração com duração máxima de 5 minutos;

- demonstrar no vídeo as funcionalidades funcionando de ponta a ponta, incluindo, sempre que possível, a interação com a interface e os resultados obtidos no sistema.

O vídeo deverá permitir a verificação objetiva do funcionamento das funcionalidades entregues.

Atenção: vídeo obrigatório

O envio do vídeo é obrigatório para a avaliação do projeto. Se o grupo não enviar o vídeo de demonstração, o projeto não será avaliado.

Serão verificados:

- o funcionamento das funcionalidades;
- a integração entre frontend e backend;
- a organização do projeto;
- a separação correta das responsabilidades;
- o uso adequado de Controller, Service, Model e Repository;
- a comunicação correta com o banco de dados;
- a coerência entre o código entregue, o diagrama de classes e a arquitetura trabalhada em aula;
- a correspondência entre as funcionalidades demonstradas no vídeo e aquelas descritas no README e disponíveis no repositório.

Atenção: ter 10 telas ou 10 rotas não significa possuir 10 funcionalidades completas. Cada funcionalidade precisa estar implementada e funcionando em todo o fluxo do sistema.